

Assessment 1 – The Physical Schema

Aplicatie e-commerce – schema ECOM_APP

Membrii echipei:

Paun Marius

Mamaishi Lucian

Ragazan Diana

Aceasta sectiune documenteaza scripturile si comenzile de testare pentru Assessment 1. Codul este bazat pe implementarea rulata in Oracle: tablespace-uri dedicate, schema separata ECOM_APP, secvente, tabele, constrangeri, LOB-uri si verificarea segmentelor fizice dupa inserarea datelor.

A1.1. Structura aplicatiei si distributia fizica

Componenta	Obiecte	Tablespace	Motivatie
Utilizatori	USERS	TBS_ECOM_USERS	Date despre conturi; PCTFREE mai mare pentru eventuale update-uri.
Catalog	CATEGORIES, PRODUCTS + CLOB	TBS_ECOM_CATALOG	Date relativ statice; contine si segmente LOB pentru descrieri.
Tranzactii	ORDERS, ORDER_ITEMS, INVOICES	TBS_ECOM_ORDERS	Date cu volum mai mare si scrieri frecvente.
Cos cumparaturi	CART_ITEMS	TBS_ECOM_CART	Date efemere, sterse/modificate frecvent.
Indexuri	PK, UNIQUE, indexuri suplimentare	TBS_ECOM_IDX	Separare fata de date pentru reducerea I/O contention.
Temporar	Sortari, hash join, group by	TBS_ECOM_TEMP	Operatii temporare Oracle.

A1.2. Pregatire directoare Linux pentru simularea discurilor

```
--pe Linux, ca root
mkdir -p /ora/disk1 /ora/disk2 /ora/disk3
chown -R oracle:oinstall /ora/disk1 /ora/disk2 /ora/disk3
chmod -R 755 /ora/disk1 /ora/disk2 /ora/disk3
ls -la /ora/
```

Rezultat asteptat: trebuie sa existe directoarele /ora/disk1, /ora/disk2 si /ora/disk3 cu owner oracle:oinstall.

Output / rezultat obtinut:

```
total 0
drwxrwxr-x 1 oracle oinstall 30 May 20 13:43 .
dr-xr-xr-x 1 root root 180 May 19 14:04 ..
drwxr-xr-x 1 oracle oinstall 100 May 20 13:53 disk1
drwxr-xr-x 1 oracle oinstall 64 May 20 13:54 disk2
drwxr-xr-x 1 oracle oinstall 60 May 20 13:54 disk3
```

A1.3. Script tablespace-uri

--Se ruleaza ca SYSDBA.

sqlplus / AS SYSDBA

```
CREATE TABLESPACE tbs_ecom_users
  DATAFILE '/ora/disk1/ecom_users01.dbf' SIZE 50M
  AUTOEXTEND ON NEXT 10M MAXSIZE 200M
  EXTENT MANAGEMENT LOCAL UNIFORM SIZE 1M
  SEGMENT SPACE MANAGEMENT AUTO
  ONLINE;

CREATE TABLESPACE tbs_ecom_catalog
  DATAFILE '/ora/disk2/ecom_catalog01.dbf' SIZE 100M
  AUTOEXTEND ON NEXT 20M MAXSIZE 500M
  EXTENT MANAGEMENT LOCAL UNIFORM SIZE 2M
  SEGMENT SPACE MANAGEMENT AUTO
  ONLINE;

CREATE TABLESPACE tbs_ecom_orders
  DATAFILE '/ora/disk1/ecom_orders01.dbf' SIZE 100M AUTOEXTEND ON NEXT 50M MAXSIZE 2G,
           '/ora/disk1/ecom_orders02.dbf' SIZE 100M AUTOEXTEND ON NEXT 50M MAXSIZE 2G
  EXTENT MANAGEMENT LOCAL UNIFORM SIZE 4M
  SEGMENT SPACE MANAGEMENT AUTO
  ONLINE;

CREATE TABLESPACE tbs_ecom_cart
  DATAFILE '/ora/disk3/ecom_cart01.dbf' SIZE 30M
  AUTOEXTEND ON NEXT 10M MAXSIZE 100M
  EXTENT MANAGEMENT LOCAL UNIFORM SIZE 512K
  SEGMENT SPACE MANAGEMENT AUTO
  ONLINE;

CREATE TABLESPACE tbs_ecom_idx
  DATAFILE '/ora/disk2/ecom_idx01.dbf' SIZE 80M
  AUTOEXTEND ON NEXT 20M MAXSIZE 400M
  EXTENT MANAGEMENT LOCAL UNIFORM SIZE 1M
  SEGMENT SPACE MANAGEMENT AUTO
  ONLINE;

CREATE TEMPORARY TABLESPACE tbs_ecom_temp
  TEMPFILE '/ora/disk3/ecom_temp01.dbf' SIZE 50M
```

```
AUTOEXTEND ON NEXT 10M MAXSIZE 200M
EXTENT MANAGEMENT LOCAL UNIFORM SIZE 1M;
```

```
SELECT tablespace_name, status, extent_management, segment_space_management
FROM dba_tablespaces
WHERE tablespace_name LIKE 'TBS_ECOM%'
ORDER BY tablespace_name;
```

Rezultat asteptat: trebuie sa apara cele 6 tablespace-uri TBS_ECOM_*, cu status ONLINE.

Output / rezultat obtinut:

TABLESPACE_NAME	STATUS	EXTENT_MAN	SEGMENT
TBS_ECOM_CART	ONLINE	LOCAL	AUTO
TBS_ECOM_CATALOG	ONLINE	LOCAL	AUTO
TBS_ECOM_IDX	ONLINE	LOCAL	AUTO
TBS_ECOM_ORDERS	ONLINE	LOCAL	AUTO
TBS_ECOM_TEMP	ONLINE	LOCAL	MANUAL
TBS_ECOM_USERS	ONLINE	LOCAL	AUTO

6 rows selected.

A1.4. Creare schema ECOM_APP si privilegii

```
CONNECT / AS SYSDBA
```

```
CREATE USER ecom_app
  IDENTIFIED BY "EcomApp_2025#"
  DEFAULT TABLESPACE tbs_ecom_orders
  TEMPORARY TABLESPACE tbs_ecom_temp
  QUOTA UNLIMITED ON tbs_ecom_users
  QUOTA UNLIMITED ON tbs_ecom_catalog
  QUOTA UNLIMITED ON tbs_ecom_orders
  QUOTA UNLIMITED ON tbs_ecom_cart
  QUOTA UNLIMITED ON tbs_ecom_idx;
```

```
GRANT CREATE SESSION TO ecom_app;
GRANT CREATE TABLE TO ecom_app;
GRANT CREATE SEQUENCE TO ecom_app;
GRANT CREATE VIEW TO ecom_app;
GRANT CREATE PROCEDURE TO ecom_app;
GRANT CREATE TRIGGER TO ecom_app;
```

Rezultat asteptat: userul ECOM_APP trebuie creat in schema separata, nu in SYS/SYSTEM.

Output / rezultat obtinut:

```
SQL>
User created.
SQL>
Grant succeeded.
SQL>
Grant succeeded.
```

```
SQL>
Grant succeeded.
SQL>
Grant succeeded.
SQL>
Grant succeeded.
SQL>
Grant succeeded.
```

```
SELECT username, account_status, default_tablespace, temporary_tablespace
FROM dba_users
WHERE username = 'ECOM_APP';
```

Rezultat asteptat: ECOM_APP trebuie sa fie OPEN, cu default tablespace TBS_ECOM_ORDERS si temporary tablespace TBS_ECOM_TEMP.

Output / rezultat obtinut:

```
SQL> SELECT username, account_status, default_tablespace, temporary_tablespace
FROM dba_users
WHERE username = 'ECOM_APP';
```

USERNAME

ACCOUNT_STATUS DEFAULT_TABLESPACE

TEMPORARY_TABLESPACE

ECOM_APP

OPEN TBS_ECOM_ORDERS

TBS_ECOM_TEMP

A1.5. Secvente pentru ID-uri

Se ruleaza conectat ca ECOM_APP. Secventele sunt obiecte separate si nu apar ca TABLE/INDEX in DBA_SEGMENTS.

```
CONNECT ecom_app/"EcomApp_2025#"
```

```
CREATE SEQUENCE seq_users_id START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE SEQUENCE seq_categories_id START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE SEQUENCE seq_products_id START WITH 1 INCREMENT BY 1 CACHE 20 NOCYCLE;
CREATE SEQUENCE seq_orders_id START WITH 1 INCREMENT BY 1 CACHE 50 NOCYCLE;
CREATE SEQUENCE seq_order_items_id START WITH 1 INCREMENT BY 1 CACHE 50 NOCYCLE;
CREATE SEQUENCE seq_cart_items_id START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
CREATE SEQUENCE seq_invoices_id START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;
```

Output / rezultat obtinut:

```
SQL>
Sequence created.
SQL>
Sequence created.
SQL>
```

```
Sequence created.
SQL>
Sequence created.
SQL>
Sequence created.
SQL>
Sequence created.
SQL>
Sequence created.
```

```
SELECT sequence_name
FROM user_sequences
ORDER BY sequence_name;
```

Rezultat asteptat: trebuie sa apara cele 7 secvente SEQ_*_ID.

Output / rezultat obtinut:

```
SQL>
SEQUENCE_NAME
-----
SEQ_CART_ITEMS_ID
SEQ_CATEGORIES_ID
SEQ_INVOICES_ID
SEQ_ORDERS_ID
SEQ_ORDER_ITEMS_ID
SEQ_PRODUCTS_ID
SEQ_USERS_ID

7 rows selected.
```

A1.6. Tabele, constrangeri si attribute de segment

```
CONNECT ecom_app/"EcomApp_2025#"
```

```
CREATE TABLE users (
    id            INTEGER DEFAULT seq_users_id.NEXTVAL,
    username      VARCHAR2(100) NOT NULL,
    email         VARCHAR2(255) NOT NULL,
    password      VARCHAR2(255) NOT NULL,
    role          VARCHAR2(50) DEFAULT 'customer',
    created_at    TIMESTAMP DEFAULT SYSTIMESTAMP,
    CONSTRAINT pk_users PRIMARY KEY (id),
    CONSTRAINT uq_users_email UNIQUE (email),
    CONSTRAINT uq_users_username UNIQUE (username),
    CONSTRAINT ck_users_role CHECK (role IN ('customer', 'admin', 'manager'))
)
TABLESPACE tbs_ecom_users
PCTFREE 20
PCTUSED 40
INITRANS 2
```

```
STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED);
```

```
CREATE TABLE categories (  
    id            INTEGER DEFAULT seq_categories_id.NEXTVAL,  
    name          VARCHAR2(200) NOT NULL,  
    description    CLOB,  
    CONSTRAINT pk_categories PRIMARY KEY (id),  
    CONSTRAINT uq_categories_name UNIQUE (name)  
)  
TABLESPACE tbs_ecom_catalog  
PCTFREE 10  
PCTUSED 60  
INITRANS 1  
STORAGE (INITIAL 512K NEXT 512K MINEXTENTS 1 MAXEXTENTS UNLIMITED)  
LOB (description) STORE AS SECUREFILE lob_categories_desc (  
    TABLESPACE tbs_ecom_catalog  
    ENABLE STORAGE IN ROW  
    COMPRESS LOW  
    CACHE  
);
```

```
CREATE TABLE products (  
    id            INTEGER DEFAULT seq_products_id.NEXTVAL,  
    name          VARCHAR2(300) NOT NULL,  
    description    CLOB,  
    price         NUMBER(10,2) NOT NULL,  
    stock_quantity INTEGER DEFAULT 0,  
    image_url      VARCHAR2(500),  
    category_id    INTEGER NOT NULL,  
    created_at     TIMESTAMP DEFAULT SYSTIMESTAMP,  
    CONSTRAINT pk_products PRIMARY KEY (id),  
    CONSTRAINT fk_products_category FOREIGN KEY (category_id) REFERENCES categories(id),  
    CONSTRAINT ck_products_price CHECK (price >= 0),  
    CONSTRAINT ck_products_stock CHECK (stock_quantity >= 0)  
)  
TABLESPACE tbs_ecom_catalog  
PCTFREE 15  
PCTUSED 50  
INITRANS 2  
STORAGE (INITIAL 2M NEXT 2M MINEXTENTS 1 MAXEXTENTS UNLIMITED)  
LOB (description) STORE AS SECUREFILE lob_products_desc (  
    TABLESPACE tbs_ecom_catalog  
    ENABLE STORAGE IN ROW  
    COMPRESS LOW  
    CACHE  
);
```

```

CREATE TABLE orders (
    id            INTEGER DEFAULT seq_orders_id.NEXTVAL,
    user_id       INTEGER NOT NULL,
    order_date    TIMESTAMP DEFAULT SYSTIMESTAMP,
    status        VARCHAR2(50) DEFAULT 'pending',
    total_amount  NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_orders PRIMARY KEY (id),
    CONSTRAINT fk_orders_user FOREIGN KEY (user_id) REFERENCES users(id),
    CONSTRAINT ck_orders_status CHECK (status IN
('pending','processing','shipped','delivered','cancelled')),
    CONSTRAINT ck_orders_amount CHECK (total_amount >= 0)
)
TABLESPACE tbs_ecom_orders
PCTFREE 25
PCTUSED 40
INITRANS 4
STORAGE (INITIAL 4M NEXT 4M MINEXTENTS 1 MAXEXTENTS UNLIMITED);

```


```

CREATE TABLE order_items (
    id            INTEGER DEFAULT seq_order_items_id.NEXTVAL,
    order_id      INTEGER NOT NULL,
    product_id    INTEGER NOT NULL,
    quantity      INTEGER NOT NULL,
    unit_price    NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_order_items PRIMARY KEY (id),
    CONSTRAINT fk_oi_order FOREIGN KEY (order_id) REFERENCES orders(id) ON DELETE CASCADE,
    CONSTRAINT fk_oi_product FOREIGN KEY (product_id) REFERENCES products(id),
    CONSTRAINT ck_oi_quantity CHECK (quantity > 0),
    CONSTRAINT ck_oi_price CHECK (unit_price >= 0)
)
TABLESPACE tbs_ecom_orders
PCTFREE 5
PCTUSED 60
INITRANS 4
STORAGE (INITIAL 4M NEXT 4M MINEXTENTS 1 MAXEXTENTS UNLIMITED);

```

-

```

CREATE TABLE cart_items (
    id            INTEGER DEFAULT seq_cart_items_id.NEXTVAL,
    user_id       INTEGER NOT NULL,
    product_id    INTEGER NOT NULL,
    quantity      INTEGER NOT NULL,
    added_at      TIMESTAMP DEFAULT SYSTIMESTAMP,
    CONSTRAINT pk_cart_items PRIMARY KEY (id),
    CONSTRAINT fk_cart_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_cart_product FOREIGN KEY (product_id) REFERENCES products(id) ON DELETE
CASCADE,
    CONSTRAINT uq_cart_user_product UNIQUE (user_id, product_id),

```

```

        CONSTRAINT ck_cart_quantity CHECK (quantity > 0)
    )
    TABLESPACE tbs_ecom_cart
    PCTFREE 10
    PCTUSED 70
    INITRANS 3
    STORAGE (INITIAL 512K NEXT 512K MINEXTENTS 1 MAXEXTENTS UNLIMITED);
-----
-----

```

```

CREATE TABLE invoices (
    id                INTEGER DEFAULT seq_invoices_id.NEXTVAL,
    order_id          INTEGER NOT NULL,
    invoice_number     VARCHAR2(50) NOT NULL,
    generated_at       TIMESTAMP DEFAULT SYSTIMESTAMP,
    total_amount       NUMBER(10,2) NOT NULL,
    CONSTRAINT pk_invoices PRIMARY KEY (id),
    CONSTRAINT fk_invoices_order FOREIGN KEY (order_id) REFERENCES orders(id),
    CONSTRAINT uq_invoices_order UNIQUE (order_id),
    CONSTRAINT uq_invoices_number UNIQUE (invoice_number),
    CONSTRAINT ck_invoices_amount CHECK (total_amount >= 0)
)
TABLESPACE tbs_ecom_orders
PCTFREE 5
PCTUSED 60
INITRANS 2
STORAGE (INITIAL 1M NEXT 1M MINEXTENTS 1 MAXEXTENTS UNLIMITED);

```

A1.7. Indexuri suplimentare

PK si UNIQUE creeaza indexuri automat. De aceea, indexurile pe email/username/invoice_number pot da ORA-01408 daca exista deja index pe aceeasi coloana. Pentru test se pastreaza indexurile suplimentare pe coloane de join/filtrare.

```
CONNECT ecom_app/"EcomApp_2025#"
```

```

CREATE INDEX idx_products_category ON products(category_id) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_products_price ON products(price) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_orders_user_id ON orders(user_id) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_orders_status ON orders(status) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_orders_date ON orders(order_date) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_oi_order_id ON order_items(order_id) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_oi_product_id ON order_items(product_id) TABLESPACE tbs_ecom_idx;
CREATE INDEX idx_cart_user_id ON cart_items(user_id) TABLESPACE tbs_ecom_idx;

```

Rezultat asteptat: indexurile trebuie create in TBS_ECOM_IDX; pentru coloanele deja indexate de PK/UNIQUE se evita dublarea.

Output / rezultat obtinut:

```

SQL>
Index created.

```



```
SQL>
Index created.
SQL>
Index created.
SQL>
Index created.
SQL>
Index created.
SQL>
Index created.
SQL>
Index created.
SQL>
Index created.
SQL>
Index created.
```

A1.8. Date demo pentru alocarea segmentelor

In Oracle, daca este activata crearea amanata a segmentelor, tabelele apar in USER_TABLES imediat, dar segmentele fizice apar in DBA_SEGMENTS dupa primul INSERT.

```
CONNECT ecom_app/"EcomApp_2025#"
```

```
INSERT INTO users (username, email, password)
VALUES ('user1', 'user1@mail.com', 'pass');
```

```
INSERT INTO categories (name, description)
VALUES ('Electronics', 'Devices');
```

```
INSERT INTO products (name, price, category_id)
VALUES ('Laptop', 3500, 1);
```

```
INSERT INTO orders (user_id, total_amount)
VALUES (1, 3500);
```

```
INSERT INTO order_items (order_id, product_id, quantity, unit_price)
VALUES (1, 1, 1, 3500);
```

```
INSERT INTO cart_items (user_id, product_id, quantity)
VALUES (1, 1, 2);
```

```
INSERT INTO invoices (order_id, invoice_number, total_amount)
VALUES (1, 'INV001', 3500);
```

```
COMMIT;
```

Output / rezultat obtinut:

```
1 row created.
```

```
SQL> SQL> 2
1 row created.
```

```
SQL> SQL> 2
```

```

1 row created.

SQL> SQL> 2
1 row created.

SQL> SQL> 2
1 row created.

SQL> SQL> 2
1 row created.

SQL> SQL> 2
1 row created.

SQL> SQL>
Commit complete.

```

A1.9. Verificari finale pentru Assessment 1

CONNECT / AS SYSDBA

```

SELECT segment_name, segment_type, tablespace_name,
       ROUND(bytes/1024/1024, 2) AS size_mb
FROM dba_segments
WHERE owner = 'ECOM_APP'
ORDER BY segment_type, segment_name;

```

Rezultat asteptat: trebuie sa apara segmente de tip TABLE, INDEX, LOBSEGMENT si LOBINDEX pentru schema ECOM_APP.

Output / rezultat obtinut:

```

SQL>
SEGMENT_NAME
-----
SEGMENT_TYPE  TABLESPACE_NAME      SIZE_MB
-----
IDX_CART_USER_ID
INDEX          TBS_ECOM_IDX          1

IDX_OI_ORDER_ID
INDEX          TBS_ECOM_IDX          1

IDX_OI_PRODUCT_ID
INDEX          TBS_ECOM_IDX          1

SEGMENT_NAME
-----
SEGMENT_TYPE  TABLESPACE_NAME      SIZE_MB
-----
IDX_ORDERS_DATE
INDEX          TBS_ECOM_IDX          1

```

IDX_ORDERS_STATUS
INDEX TBS_ECOM_IDX 1

IDX_ORDERS_USER_ID
INDEX TBS_ECOM_IDX 1

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

IDX_PRODUCTS_CATEGORY
INDEX TBS_ECOM_IDX 1

IDX_PRODUCTS_PRICE
INDEX TBS_ECOM_IDX 1

PK_CART_ITEMS
INDEX TBS_ECOM_CART .5

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

PK_CATEGORIES
INDEX TBS_ECOM_CATALOG 2

PK_INVOICES
INDEX TBS_ECOM_ORDERS 4

PK_ORDERS
INDEX TBS_ECOM_ORDERS 4

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

PK_ORDER_ITEMS
INDEX TBS_ECOM_ORDERS 4

PK_PRODUCTS
INDEX TBS_ECOM_CATALOG 2

PK_USERS
INDEX TBS_ECOM_USERS 1

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

UQ_CART_USER_PRODUCT
INDEX TBS_ECOM_CART .5

UQ_CATEGORIES_NAME
INDEX TBS_ECOM_CATALOG 2

UQ_INVOICES_NUMBER
INDEX TBS_ECOM_ORDERS 4

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

UQ_INVOICES_ORDER
INDEX TBS_ECOM_ORDERS 4

UQ_USERS_EMAIL
INDEX TBS_ECOM_USERS 1

UQ_USERS_USERNAME
INDEX TBS_ECOM_USERS 1

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

SYS_IL0000073942C00003\$\$
LOBINDEX TBS_ECOM_CATALOG 2

SYS_IL0000073947C00003\$\$
LOBINDEX TBS_ECOM_CATALOG 2

LOB_CATEGORIES_DESC
LOBSEGMENT TBS_ECOM_CATALOG 2

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

LOB_PRODUCTS_DESC
LOBSEGMENT TBS_ECOM_CATALOG 2

CART_ITEMS
TABLE TBS_ECOM_CART .5

CATEGORIES
TABLE TBS_ECOM_CATALOG 2

SEGMENT_NAME

SEGMENT_TYPE TABLESPACE_NAME SIZE_MB

INVOICES
TABLE TBS_ECOM_ORDERS 4

```

ORDERS
TABLE      TBS_ECOM_ORDERS      4

ORDER_ITEMS
TABLE      TBS_ECOM_ORDERS      4

SEGMENT_NAME
-----
SEGMENT_TYPE  TABLESPACE_NAME      SIZE_MB
-----
PRODUCTS
TABLE      TBS_ECOM_CATALOG      2

USERS
TABLE      TBS_ECOM_USERS      1

32 rows selected.

```

```

SELECT tablespace_name, COUNT(*) AS nr_segmente,
       ROUND(SUM(bytes)/1024/1024, 2) AS total_mb
FROM dba_segments
WHERE owner = 'ECOM_APP'
GROUP BY tablespace_name
ORDER BY tablespace_name;

```

Rezultat asteptat: trebuie sa se vada distributia pe TBS_ECOM_USERS, TBS_ECOM_CATALOG, TBS_ECOM_ORDERS, TBS_ECOM_CART si TBS_ECOM_IDX.

Output / rezultat obtinut:

```

SQL> SELECT tablespace_name, COUNT(*) AS nr_segmente,
       ROUND(SUM(bytes)/1024/1024, 2) AS total_mb
FROM dba_segments
WHERE owner = 'ECOM_APP'
GROUP BY tablespace_name
ORDER BY tablespace_name;
 2  3  4  5  6
TABLESPACE_NAME      NR_SEGMENTS  TOTAL_MB
-----
TBS_ECOM_CART        3        1.5
TBS_ECOM_CATALOG     9        18
TBS_ECOM_IDX         8         8
TBS_ECOM_ORDERS      8        32
TBS_ECOM_USERS       4         4

```

```

SELECT tablespace_name, file_name,
       ROUND(bytes/1024/1024) AS size_mb,
       autoextensible
FROM dba_data_files
WHERE tablespace_name LIKE 'TBS_ECOM%'
ORDER BY tablespace_name, file_name;

```

Rezultat asteptat: trebuie sa se vada datafile-urile din /ora/disk1, /ora/disk2 si /ora/disk3, cu AUTOEXTENSIBLE = YES.

Output / rezultat obtinut:

SQL>

TABLESPACE_NAME

FILE_NAME

SIZE_MB AUT

TBS_ECOM_CART

/ora/disk3/ecom_cart01.dbf

30 YES

TBS_ECOM_CATALOG

/ora/disk2/ecom_catalog01.dbf

100 YES

TABLESPACE_NAME

FILE_NAME

SIZE_MB AUT

TBS_ECOM_IDX

/ora/disk2/ecom_idx01.dbf

80 YES

TBS_ECOM_ORDERS

/ora/disk1/ecom_orders01.dbf

TABLESPACE_NAME

FILE_NAME

SIZE_MB AUT

100 YES

TBS_ECOM_ORDERS

/ora/disk1/ecom_orders02.dbf

100 YES

TBS_ECOM_USERS

TABLESPACE_NAME

FILE_NAME

SIZE_MB AUT

/ora/disk1/ecom_users01.dbf

50 YES

6 rows selected.

```
SELECT constraint_name, constraint_type, table_name, status
FROM dba_constraints
WHERE owner = 'ECOM_APP'
ORDER BY table_name, constraint_type, constraint_name;
```

Rezultat asteptat: trebuie sa apara constrangeri P=Primary Key, R=Foreign Key, U=Unique si C=Check, toate ENABLED.

Output / rezultat obtinut:

SQL>

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_CART_QUANTITY

C

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007663

C

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007664

C

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007665

C

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_CART_ITEMS

P

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_CART_PRODUCT

R

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_CART_USER

R

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

UQ_CART_USER_PRODUCT

U

CART_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007638

C

CATEGORIES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_CATEGORIES

P

CATEGORIES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

UQ_CATEGORIES_NAME

U

CATEGORIES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_INVOICES_AMOUNT

C

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007671

C

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007672

C

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007673

C

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_INVOICES

P

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_INVOICES_ORDER

R

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

UQ_INVOICES_NUMBER

U

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

UQ_INVOICES_ORDER

U

INVOICES

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_ORDERS_AMOUNT

C

ORDERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_ORDERS_STATUS

C

ORDERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007648

C

ORDERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007649

C

ORDERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_ORDERS

P

ORDERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_ORDERS_USER

R

ORDERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_OI_PRICE

C

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_OI_QUANTITY

C

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007654

C

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007655

C

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007656

C

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007657

C

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_ORDER_ITEMS

P

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_OI_ORDER

R

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_OI_PRODUCT

R

ORDER_ITEMS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_PRODUCTS_PRICE

C

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_PRODUCTS_STOCK

C

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007641

C

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007642

C

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007643

C

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_PRODUCTS

P

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

FK_PRODUCTS_CATEGORY

R

PRODUCTS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

CK_USERS_ROLE

C

USERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007631

C

USERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007632

C

USERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

SYS_C007633

C

USERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

PK_USERS

P

USERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

UQ_USERS_EMAIL

U

USERS

ENABLED

CONSTRAINT_NAME

C

-

TABLE_NAME

STATUS

UQ_USERS_USERNAME

U

USERS

ENABLED

48 rows selected.

```
SELECT segment_type, COUNT(*)
FROM dba_segments
WHERE owner = 'ECOM_APP'
GROUP BY segment_type
ORDER BY segment_type;
```

Rezultat asteptat: in rularea curenta au rezultat: TABLE = 7, INDEX = 21, LOBSEGMENT = 2, LOBINDEX = 2.

Output / rezultat obtinut:

SQL>

SEGMENT_TYPE	COUNT(*)
--------------	----------

INDEX	21
-------	----

LOBINDEX	2
----------	---

LOBSEGMENT	2
------------	---

TABLE	7
-------	---

A1.10. Interpretare rezultate pentru prezentare

- Schema aplicatiei este separata in userul ECOM_APP, nu in SYS/SYSTEM.
- Tablespace-urile sunt specifice aplicatiei si separa datele pe tipuri: users, catalog, orders, cart, index si temp.
- Fisierle sunt plasate in directoare diferite pentru a simula distributia pe discuri diferite: /ora/disk1, /ora/disk2 si /ora/disk3.
- Parametrii PCTFREE, PCTUSED, INITRANS si STORAGE au fost alesi in functie de tipul datelor si frecventa update-urilor.
- CLOB-urile din CATEGORIES si PRODUCTS creeaza segmente LOB separate, deci schema contine mai mult decat TABLE si INDEX.

- După inserarea datelor, DBA_SEGMENTS confirmă alocarea fizică a segmentelor în tablespace-urile proiectate.

Week 14 – Assessment 3

Security, Audit, Backup & Recovery

Proiect Oracle DBA – Aplicatie E-commerce / Schema ECOM_APP

0. Impartirea muncii pentru 3 persoane

Persoana	Componenta	Ce demonstreaza
Persoana 1	Security	Roluri Oracle, useri de test, privilegii si scenarii read/write.
Persoana 2	Audit	Audit pentru userul ECOM_MANAGER_U si verificare in DBA_AUDIT_TRAIL.
Paun Marius	Backup & Recovery	Strategie RPO/RTO, backup RMAN si scenariu de recuperare.

1. Verificari initiale

Aceste comenzi confirma ca baza de date, schema aplicatiei si tabelele exista inainte de testarea securitatii.

```
CONNECT / AS SYSDBA
SHOW USER;
```

```
SELECT username, account_status, default_tablespace, temporary_tablespace
FROM dba_users
WHERE username IN ('ECOM_APP', 'ECOM_CUSTOMER_U', 'ECOM_MANAGER_U', 'ECOM_ADMIN_U')
ORDER BY username;
```

Output / rezultat obtinut:

```
SQL>
```

```
USERNAME
```

```
-----
ACCOUNT_STATUS      DEFAULT_TABLESPACE
```

```
-----
TEMPORARY_TABLESPACE
```

```
-----
ECOM_APP
```

```
OPEN              TBS_ECOM_ORDERS
```

```
TBS_ECOM_TEMP
```

```
SELECT table_name
FROM all_tables
WHERE owner = 'ECOM_APP'
ORDER BY table_name;
```

Rezultat asteptat: trebuie sa apara cele 7 tabele: USERS, CATEGORIES, PRODUCTS, ORDERS, ORDER_ITEMS, CART_ITEMS, INVOICES.

Output / rezultat obtinut:

SQL>

TABLE_NAME

```
-----  
CART_ITEMS  
CATEGORIES  
INVOICES  
ORDERS  
ORDER_ITEMS  
PRODUCTS  
USERS
```

7 rows selected.

2. Security – roluri, useri si privilegii

Scriptul se ruleaza ca SYSDBA. Daca rolurile/userii exista deja, se pot ignora erorile de tip „already exists” sau se poate rula intai partea de DROP.

```
CONNECT / AS SYSDBA
```

```
CREATE ROLE role_ecom_customer;  
CREATE ROLE role_ecom_manager;  
CREATE ROLE role_ecom_admin;
```

```
CREATE USER ecom_customer_u IDENTIFIED BY "Customer_2025#"   
DEFAULT TABLESPACE tbs_ecom_cart  
TEMPORARY TABLESPACE tbs_ecom_temp  
QUOTA 20M ON tbs_ecom_cart;
```

```
CREATE USER ecom_manager_u IDENTIFIED BY "Manager_2025#"   
DEFAULT TABLESPACE tbs_ecom_orders  
TEMPORARY TABLESPACE tbs_ecom_temp  
QUOTA 50M ON tbs_ecom_orders  
QUOTA 50M ON tbs_ecom_catalog;
```

```
CREATE USER ecom_admin_u IDENTIFIED BY "Admin_2025#"   
DEFAULT TABLESPACE tbs_ecom_orders  
TEMPORARY TABLESPACE tbs_ecom_temp  
QUOTA UNLIMITED ON tbs_ecom_users  
QUOTA UNLIMITED ON tbs_ecom_catalog  
QUOTA UNLIMITED ON tbs_ecom_orders  
QUOTA UNLIMITED ON tbs_ecom_cart  
QUOTA UNLIMITED ON tbs_ecom_idx;
```

```
GRANT CREATE SESSION TO ecom_customer_u;  
GRANT CREATE SESSION TO ecom_manager_u;  
GRANT CREATE SESSION TO ecom_admin_u;
```

Output / rezultat obtinut:

SQL>

Role created.

```
SQL>
Role created.
SQL>
Role created.

SQL>
User created.
SQL>
User created.
SQL>
User created.

SQL>
Grant succeeded.
SQL>
Grant succeeded.
SQL>
Grant succeeded.
```

CONNECT / AS SYSDBA

```
-- Customer: citire catalog + lucru cu cos/comenzi
GRANT SELECT ON ecom_app.categories TO role_ecom_customer;
GRANT SELECT ON ecom_app.products TO role_ecom_customer;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.cart_items TO role_ecom_customer;
GRANT SELECT, INSERT ON ecom_app.orders TO role_ecom_customer;
GRANT SELECT, INSERT ON ecom_app.order_items TO role_ecom_customer;
GRANT SELECT ON ecom_app.invoices TO role_ecom_customer;

-- Manager: citire date operationale + update pe produse/comenzi
GRANT SELECT ON ecom_app.users TO role_ecom_manager;
GRANT SELECT, INSERT, UPDATE ON ecom_app.categories TO role_ecom_manager;
GRANT SELECT, INSERT, UPDATE ON ecom_app.products TO role_ecom_manager;
GRANT SELECT, UPDATE ON ecom_app.orders TO role_ecom_manager;
GRANT SELECT ON ecom_app.order_items TO role_ecom_manager;
GRANT SELECT ON ecom_app.cart_items TO role_ecom_manager;
GRANT SELECT ON ecom_app.invoices TO role_ecom_manager;

-- Admin: acces complet pe datele aplicatiei
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.users TO role_ecom_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.categories TO role_ecom_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.products TO role_ecom_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.orders TO role_ecom_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.order_items TO role_ecom_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.cart_items TO role_ecom_admin;
GRANT SELECT, INSERT, UPDATE, DELETE ON ecom_app.invoices TO role_ecom_admin;

GRANT role_ecom_customer TO ecom_customer_u;
GRANT role_ecom_manager TO ecom_manager_u;
GRANT role_ecom_admin TO ecom_admin_u;

ALTER USER ecom_customer_u DEFAULT ROLE role_ecom_customer;
```

```
ALTER USER ecom_manager_u DEFAULT ROLE role_ecom_manager;
ALTER USER ecom_admin_u DEFAULT ROLE role_ecom_admin;
```

Output / rezultat obtinut:

```
SQL>
Grant succeeded.
SQL>
Grant succeeded.
SQL>
Grant succeeded.
....
SQL>
Grant succeeded.

SQL>
User altered.
SQL>
User altered.
SQL>
User altered.
```

3. Test rol CUSTOMER

```
CONNECT ecom_customer_u/"Customer_2025#"
SHOW USER;
SELECT * FROM session_roles;
```

Rezultat asteptat: userul conectat este ECOM_CUSTOMER_U si rolul activ este ROLE_ECOM_CUSTOMER.

Output / rezultat obtinut:

```
ROLE
-----
ROLE_ECOM_CUSTOMER
```

```
SELECT id, name, price
FROM ecom_app.products;
```

Rezultat asteptat: SELECT trebuie sa functioneze, deoarece clientul poate vizualiza catalogul de produse.

Output / rezultat obtinut:

```
SQL>
  ID
-----
NAME
-----
  PRICE
-----
    1
Laptop
 3500
```



```
INSERT INTO ecom_app.cart_items (user_id, product_id, quantity)
VALUES (1, 1, 1);
```

COMMIT;

Rezultat asteptat: INSERT trebuie sa functioneze daca nu exista deja combinatia user_id=1 si product_id=1. Daca apare ORA-00001, se testeaza UPDATE pe aceeasi linie.

Output / rezultat obtinut:
-(exista deja combinatia)

```
UPDATE ecom_app.cart_items
SET quantity = quantity + 1
WHERE user_id = 1
AND product_id = 1;
```

COMMIT;

Rezultat asteptat: UPDATE trebuie sa functioneze, deoarece rolul customer are drept de UPDATE pe CART_ITEMS.

Output / rezultat obtinut:
SQL>
1 row updated.
SQL>
Commit complete.

```
DELETE FROM ecom_app.products
WHERE id = 1;
```

Rezultat asteptat: trebuie sa esueze cu ORA-01031: insufficient privileges. Acest rezultat demonstreaza ca un client nu poate sterge produse.

Output / rezultat obtinut:
ERROR at line 1:
ORA-01031: insufficient privileges

4. Test rol MANAGER

```
CONNECT ecom_manager_u/"Manager_2025#"
SHOW USER;
SELECT * FROM session_roles;
```

Rezultat asteptat: userul conectat este ECOM_MANAGER_U si rolul activ este ROLE_ECOM_MANAGER.

Output / rezultat obtinut:

SQL>
Connected.
SQL>

ROLE

ROLE_ECOM_MANAGER

```
SELECT id, username, email
FROM ecom_app.users;
```

Rezultat asteptat: SELECT trebuie sa functioneze, deoarece managerul poate consulta utilizatorii.

Output / rezultat obtinut:

SQL>

ID

USERNAME

EMAIL

1

user1

user1@mail.com

```
UPDATE ecom_app.products
SET stock_quantity = stock_quantity + 10
WHERE id = 1;
```

COMMIT;

Rezultat asteptat: UPDATE trebuie sa functioneze, deoarece managerul poate modifica stocul produselor.

Output / rezultat obtinut:

SQL>

1 row updated.

SQL>

Commit complete.

```
UPDATE ecom_app.orders
SET status = 'processing'
WHERE id = 1;
```

COMMIT;

Rezultat asteptat: UPDATE trebuie sa functioneze, deoarece managerul poate modifica statusul comenzilor.

Output / rezultat obtinut:

SQL>

1 row updated.

SQL>

Commit complete.

```
DELETE FROM ecom_app.users
WHERE id = 1;
```

Rezultat asteptat: trebuie sa esueze cu ORA-01031: insufficient privileges. Managerul nu are drept de DELETE pe USERS.

Output / rezultat obtinut:

```
SQL>
ERROR at line 1:
ORA-01031: insufficient privileges
```

5. Test rol ADMIN

```
CONNECT ecom_admin_u/"Admin_2025#"
SHOW USER;
SELECT * FROM session_roles;
```

Rezultat asteptat: userul conectat este ECOM_ADMIN_U si rolul activ este ROLE_ECOM_ADMIN.

Output / rezultat obtinut:

```
ROLE
-----
ROLE_ECOM_ADMIN
```

```
SELECT COUNT(*) AS nr_comenzi
FROM ecom_app.orders;
```

Rezultat asteptat: SELECT trebuie sa functioneze.

Output / rezultat obtinut:

```
SQL>
NR_COMENZI
-----
1
```

```
UPDATE ecom_app.orders
SET status = 'delivered'
WHERE id = 1;
```

```
COMMIT;
```

Rezultat asteptat: UPDATE trebuie sa functioneze, deoarece adminul are acces complet pe tabelele aplicatiei.

Output / rezultat obtinut:

```
SQL>
1 row updated.
SQL>
Commit complete.
```

6. Audit pentru ECOM_MANAGER_U

Strategia de audit: auditam userul ECOM_MANAGER_U deoarece poate modifica preturi, stocuri si statusuri de comenzi. Aceste actiuni sunt sensibile pentru o aplicatie e-commerce.

```
CONNECT / AS SYSDBA
```

```
AUDIT SESSION BY ecom_manager_u;  
AUDIT INSERT, UPDATE, DELETE ON ecom_app.products BY ACCESS;  
AUDIT UPDATE ON ecom_app.orders BY ACCESS;
```

Output / rezultat obtinut:

```
SQL>  
Audit succeeded.  
SQL>  
Audit succeeded.  
SQL>  
Audit succeeded.
```

```
CONNECT ecom_manager_u/"Manager_2025#"
```

```
UPDATE ecom_app.products  
SET price = price + 100  
WHERE id = 1;
```

```
UPDATE ecom_app.orders  
SET status = 'processing'  
WHERE id = 1;
```

```
COMMIT;
```

Rezultat asteptat: comenzile trebuie sa functioneze si sa genereze inregistrari de audit.

Output / rezultat obtinut:

```
SQL>  
1 row updated.  
  
SQL>  
1 row updated.  
  
SQL> SQL>  
Commit complete.
```

```
CONNECT / AS SYSDBA
```

```
SELECT username, obj_name, action_name, returncode, timestamp  
FROM dba_audit_trail  
WHERE username = 'ECOM_MANAGER_U'  
ORDER BY timestamp DESC;
```

Rezultat asteptat: trebuie sa apara actiuni de tip LOGON/SESSION si UPDATE pe PRODUCTS/ORDERS, cu RETURNCODE = 0 pentru succes.

Output / rezultat obtinut:		
SQL>		
USERNAME		

OBJ_NAME		

ACTION_NAME	RETURNCODE	TIMESTAMP

ECOM_MANAGER_U		
LOGOFF	0	21-MAY-26
ECOM_MANAGER_U		
ORDERS		
UPDATE	0	21-MAY-26
USERNAME		

OBJ_NAME		

ACTION_NAME	RETURNCODE	TIMESTAMP

ECOM_MANAGER_U		
LOGON	0	21-MAY-26
ECOM_MANAGER_U		
PRODUCTS		
USERNAME		

OBJ_NAME		

ACTION_NAME	RETURNCODE	TIMESTAMP

UPDATE	0	21-MAY-26

7. Backup strategy – RPO/RTO

Strategie propusa: backup complet RMAN al bazei de date si al control file-ului, salvat in /ora/backup/ecom.

Indicator	Valoare propusa	Justificare
RPO	maximum 24 ore	Se accepta pierderea datelor introduse dupa ultimul backup zilnic, fiind un proiect academic.
RTO	maximum 1 ora	Restore/recover se poate realiza rapid in VM, folosind backup local RMAN.

```
-- Linux, ca root
mkdir -p /ora/backup/ecom
chown -R oracle:oinstall /ora/backup
chmod -R 755 /ora/backup
```

Output / rezultat obtinut:

```
[root@orasrv ~]# mkdir -p /ora/backup/ecom
[root@orasrv ~]# chown -R oracle:oinstall /ora/backup
[root@orasrv ~]# chmod -R 755 /ora/backup
[root@orasrv ~]# ls /ora
backup disk1 disk2 disk3
[root@orasrv ~]#
```

```
-- Fisier: backup_ecom.rman
--root user
cat > /ora/backup/ecom/backup_ecom.rman << 'EOF'
RUN {
    ALLOCATE CHANNEL c1 DEVICE TYPE DISK;
    BACKUP DATABASE FORMAT '/ora/backup/ecom/full_db_%U.bkp';
    BACKUP CURRENT CONTROLFILE FORMAT '/ora/backup/ecom/controlfile_%U.bkp';
    RELEASE CHANNEL c1;
}
EOF
```

```
-- Linux, ca oracle
rman target / @backup_ecom.rman
```

Rezultat asteptat: RMAN trebuie sa finalizeze backupul fara erori.

Output / rezultat obtinut:

```
[oracle@orasrv ~]$ rman target / @/ora/backup/ecom/backup_ecom.rman

Recovery Manager: Release 12.2.0.1.0 - Production on Thu May 21 04:12:54 2026

Copyright (c) 1982, 2017, Oracle and/or its affiliates. All rights reserved.

connected to target database: MYDB (DBID=2882446206)

RMAN> RUN {
2>  ALLOCATE CHANNEL c1 DEVICE TYPE DISK;
3>  BACKUP DATABASE FORMAT '/ora/backup/ecom/full_db_%U.bkp';
4>  BACKUP CURRENT CONTROLFILE FORMAT '/ora/backup/ecom/controlfile_%U.bkp';
5>  RELEASE CHANNEL c1;
6> }
7>
using target database control file instead of recovery catalog
allocated channel: c1
channel c1: SID=62 device type=DISK
```

```

Starting backup at 21-MAY-26
channel c1: starting full datafile backup set
channel c1: specifying datafile(s) in backup set
input datafile file number=00001 name=/u01/app/oracle/oradata/MYDB/datafile/o1_mf_system_f7oyq4g1_.dbf
input datafile file number=00002 name=/ora/disk2/ecom_catalog01.dbf
input datafile file number=00008 name=/ora/disk1/ecom_orders01.dbf
input datafile file number=00009 name=/ora/disk1/ecom_orders02.dbf
input datafile file number=00011 name=/ora/disk2/ecom_idx01.dbf
input datafile file number=00005 name=/ora/disk1/ecom_users01.dbf
input datafile file number=00010 name=/ora/disk3/ecom_cart01.dbf
input datafile file number=00003 name=/u01/app/oracle/oradata/MYDB/datafile/o1_mf_sysaux_f7oyrkqk_.dbf
channel c1: starting piece 1 at 21-MAY-26
channel c1: finished piece 1 at 21-MAY-26
piece handle=/ora/backup/ecom/full_db_034okpq8_1_1.bkp tag=TAG20260521T041256 comment=NONE
channel c1: backup set complete, elapsed time: 00:00:16
channel c1: starting full datafile backup set
channel c1: specifying datafile(s) in backup set
input datafile file number=00004 name=/u01/app/oracle/oradata/MYDB/datafile/o1_mf_undotbs1_f7oyrc2g_.dbf
input datafile file number=00007 name=/u01/app/oracle/oradata/MYDB/datafile/o1_mf_users_f7oyrscv_.dbf
channel c1: starting piece 1 at 21-MAY-26
channel c1: finished piece 1 at 21-MAY-26
piece handle=/ora/backup/ecom/full_db_044okpqo_1_1.bkp tag=TAG20260521T041256 comment=NONE
channel c1: backup set complete, elapsed time: 00:00:01
Finished backup at 21-MAY-26

```

```

Starting backup at 21-MAY-26
channel c1: starting full datafile backup set
channel c1: specifying datafile(s) in backup set
including current control file in backup set
channel c1: starting piece 1 at 21-MAY-26
channel c1: finished piece 1 at 21-MAY-26
piece handle=/ora/backup/ecom/controlfile_054okpqq_1_1.bkp tag=TAG20260521T041314 comment=NONE
channel c1: backup set complete, elapsed time: 00:00:01
Finished backup at 21-MAY-26

```

```

Starting Control File and SPFILE Autobackup at 21-MAY-26
piece
handle=/u01/app/oracle/fast_recovery_area/mydb/MYDB/autobackup/2026_05_21/o1_mf_s_1233807196_o0xhnxb_.
bkp comment=NONE
Finished Control File and SPFILE Autobackup at 21-MAY-26

```

released channel: c1

Recovery Manager complete.

```
ls -lh /ora/backup/ecom
```

Rezultat asteptat: trebuie sa fie vizibile fisierele .bkp generate de RMAN.

Output / rezultat obtinut:

```

[oracle@orasrv ~]$ ls -lh /ora/backup/ecom
total 1.1G
-rw-r--r-- 1 root  root   213 May 21 04:11 backup_ecom.rman
-rw-r----- 1 oracle oinstall 11M May 21 04:13 controlfile_054okpqq_1_1.bkp

```

```
-rw-r----- 1 oracle oinstall 1.1G May 21 04:13 full_db_034okpq8_1_1.bkp
-rw-r----- 1 oracle oinstall 14M May 21 04:13 full_db_044okpqo_1_1.bkp
```

```
rman target /
```

```
LIST BACKUP SUMMARY;
```

Rezultat asteptat: trebuie sa apara backupul complet si control file backup.

Output / rezultat obtinut:

```
[oracle@orasrv ~]$ rman target /
IST BACKUP SUMMARY;

Recovery Manager: Release 12.2.0.1.0 - Production on Thu May 21 04:15:57 2026

Copyright (c) 1982, 2017, Oracle and/or its affiliates. All rights reserved.

connected to target database: MYDB (DBID=2882446206)

RMAN>
using target database control file instead of recovery catalog

List of Backups
=====
Key    TY LV S Device Type Completion Time #Pieces #Copies Compressed Tag
-----
1      B F A DISK    20-MAY-26    1     1    NO    TAG20260520T140154
2      B F A DISK    21-MAY-26    1     1    NO    TAG20260521T025209
3      B F A DISK    21-MAY-26    1     1    NO    TAG20260521T041256
4      B F A DISK    21-MAY-26    1     1    NO    TAG20260521T041256
5      B F A DISK    21-MAY-26    1     1    NO    TAG20260521T041314
6      B F A DISK    21-MAY-26    1     1    NO    TAG20260521T041316

RMAN>
```

8. Crash scenario si recovery

Scenariu: datafile-ul pentru TBS_ECOM_ORDERS este sters accidental. Se restaureaza datafile-ul din backup RMAN si se aplica recovery.

```
-- Linux, ca oracle/root:
rm /ora/disk1/ecom_orders01.dbf
```

Output / rezultat obtinut:

```
[oracle@orasrv ~]$ rm /ora/disk1/ecom_orders01.dbf
[oracle@orasrv ~]$ ls -l /ora/disk1/ecom_orders01.dbf
ls: cannot access /ora/disk1/ecom_orders01.dbf: No such file or directory
[oracle@orasrv ~]$
```



```
CONNECT / AS SYSDBA
SHUTDOWN IMMEDIATE;
STARTUP;
```

Rezultat asteptat: baza poate returna ORA-01157 / ORA-01110 pentru datafile lipsa.

Output / rezultat obtinut:

```
SQL> SHUTDOWN IMMEDIATE;
STARTUP;
```

```
ORA-01116: error in opening database file 8
ORA-01110: data file 8: '/ora/disk1/ecom_orders01.dbf'
ORA-27041: unable to open file
Linux-x86_64 Error: 2: No such file or directory
Additional information: 3
SQL> ORA-01081: cannot start already-running ORACLE - shut it down first
```

rman target /

```
STARTUP MOUNT;
RESTORE DATAFILE '/ora/disk1/ecom_orders01.dbf';
RECOVER DATAFILE '/ora/disk1/ecom_orders01.dbf';
ALTER DATABASE OPEN;
```

Rezultat asteptat: datafile-ul este restaurat, recovery-ul se finalizeaza, iar baza se deschide.

Output / rezultat obtinut:

```
[oracle@orasrv ~]$ rman target /
```

```
Recovery Manager: Release 12.2.0.1.0 - Production on Thu May 21 05:17:24 2026
```

```
Copyright (c) 1982, 2017, Oracle and/or its affiliates. All rights reserved.
```

```
connected to target database: MYDB (DBID=2882446206, not open)
```

```
RMAN> STARTUP MOUNT;
```

```
R
```

```
ESTORE DATAFILE '/ora/disk1/ecom_orders01.dbf';
RECOVER DATAFILE '/ora/disk1/ecom_orders01.dbf';
ALTER DATABASE OPEN;
database is already started
```

```
RMAN>
```

```
Starting restore at 21-MAY-26
using target database control file instead of recovery catalog
allocated channel: ORA_DISK_1
channel ORA_DISK_1: SID=41 device type=DISK
```

```
channel ORA_DISK_1: starting datafile backup set restore
channel ORA_DISK_1: specifying datafile(s) to restore from backup set
channel ORA_DISK_1: restoring datafile 00008 to /ora/disk1/ecom_orders01.dbf
channel ORA_DISK_1: reading from backup piece /ora/backup/ecom/full_db_034okpq8_1_1.bkp
channel ORA_DISK_1: piece handle=/ora/backup/ecom/full_db_034okpq8_1_1.bkp tag=TAG20260521T041256
```

```
channel ORA_DISK_1: restored backup piece 1
channel ORA_DISK_1: restore complete, elapsed time: 00:00:01
Finished restore at 21-MAY-26
```

```
RMAN>
```

```
Starting recover at 21-MAY-26
using channel ORA_DISK_1
```

```
starting media recovery
media recovery complete, elapsed time: 00:00:00
```

```
Finished recover at 21-MAY-26
```

```
RMAN>
```

```
CONNECT ecom_app/"EcomApp_2025#"
```

```
SELECT COUNT(*) FROM orders;
SELECT COUNT(*) FROM order_items;
SELECT COUNT(*) FROM invoices;
```

Rezultat asteptat: datele din tabelele tranzactionale trebuie sa fie accesibile dupa recovery.

Output / rezultat obtinut:

```
COUNT(*)
```

```
-----
      1
```

```
SQL>
```

```
COUNT(*)
```

```
-----
      1
```

```
SQL>
```

```
COUNT(*)
```

```
-----
      1
```

9. Checklist final pentru prezentare

- Rolurile ROLE_ECOM_CUSTOMER, ROLE_ECOM_MANAGER si ROLE_ECOM_ADMIN exista.
- Exista cate un user de test pentru fiecare rol: ECOM_CUSTOMER_U, ECOM_MANAGER_U, ECOM_ADMIN_U.
- Testele pozitive confirma operatiile permise.
- Testele negative confirma operatiile interzise prin ORA-01031.
- Auditul pentru ECOM_MANAGER_U apare in DBA_AUDIT_TRAIL.
- Backupul RMAN este creat in /ora/backup/ecom.
- Scenariul de crash/recovery este documentat cu pasi clari si output.